

Hello,

We are building **Smartbeeps**, a scalable CPaaS platform designed to handle high-volume messaging (1+ Crore SMS/month and growing). We require a production-ready system supporting **SMS, WhatsApp, and RCS** with multi-vendor routing and failover.

This is not a basic reseller panel. We need a strong foundation to scale to enterprise traffic levels.

Below is the complete structured requirement.

1. PROJECT OBJECTIVE

Develop a scalable, secure CPaaS platform with:

- Super Admin, Admin & Client panels
 - Public APIs for SMS, WhatsApp & RCS
 - Multi-vendor routing & failover
 - Queue-based architecture
 - Real-time DLR tracking
 - Wallet & billing system
 - DLT compliance support
 - Enterprise-grade security
-

2. USER ROLES

A) Super Admin

- Create/manage Admins
- View global volumes & revenue
- Vendor management
- Route configuration
- System monitoring
- Access to all logs

B) Admin

- Create/manage clients
- Assign API keys
- Map PE ID / Headers / Templates
- Set pricing per route
- Configure vendor priority & TPS

- View client analytics
- Monitor vendor health
- Access raw logs

C) Client

- Dashboard with analytics
- Send SMS / WhatsApp / RCS
- Bulk upload (CSV/XLSX)
- Campaign scheduling
- Download reports
- View balance
- API documentation access

3. AUTHENTICATION & SECURITY

- Username + Password login
- Optional OTP-based 2FA
- JWT authentication for APIs
- API key management
- IP whitelisting
- HTTPS mandatory
- Webhook authentication tokens
- Role-based access control
- Audit logs

4. DASHBOARD & ANALYTICS

Analytics view for:

- Today
- Yesterday
- Last 7 days
- Current month
- Custom date range

Visual reports:

- Submitted / Delivered / Failed
- Delivery percentage
- Campaign performance
- Vendor performance

- WhatsApp message status (sent/read)
- RCS interaction stats

Filters:

- Date
 - Sender ID
 - Template
 - Campaign
 - User
-

5. CHANNEL MODULES

A) SMS MODULE

Features:

- Single SMS
- Bulk upload
- Campaign creation
- Template mapping (DLT)
- Sender ID selection
- OTP API
- URL shortening
- Scheduling
- Real-time DLR tracking

Required APIs:

POST /v1/sms/send

POST /v1/sms/bulk

GET /v1/sms/report

GET /v1/balance

Status standardization:

queued / sent / delivered / failed / rejected / expired

B) WHATSAPP MODULE

Features:

- WhatsApp Business API integration
- Template message sending
- Session message sending
- Media support (image, PDF, video)
- Campaign scheduling
- Conversation tracking

- Read receipts
- Failed reason logging

Required APIs:

POST /v1/whatsapp/send

GET /v1/whatsapp/report

WhatsApp features:

- Template approval tracking
- WABA mapping
- Phone number mapping
- Conversation-based billing support
- Vendor webhook integration

C) RCS MODULE

Features:

- Rich message support (cards, carousels, buttons)
- Branding profile management
- Read receipts
- Interaction tracking (click, open)
- Campaign scheduling

Required APIs:

POST /v1/rcs/send

GET /v1/rcs/report

Must support:

- RCS template management
- Vendor webhook handling
- Event tracking (delivered, read, clicked)

6. BACKEND ARCHITECTURE REQUIREMENTS

System must include:

A) API Layer

- Public REST APIs
- Rate limiting
- API key validation
- Standard JSON responses
- Channel abstraction (SMS/WA/RCS unified logic)

B) Queue-Based Processing

- Redis integration
- API pushes message to queue
- Worker processes queue
- Burst traffic handling
- Retry mechanism

C) Multi-Vendor Integration

- Vendor A (Primary)
- Vendor B (Secondary)
- Priority-based routing
- 80/20 load splitting option
- Automatic failover on API failure
- Health monitoring

D) TPS Throttling

- Respect vendor TPS limits
- Controlled dispatch via worker

E) Vendor Health Engine

- Monitor API error rate
- Monitor timeout rate
- Auto-mark vendor unhealthy
- Auto-recover after cooldown

7. DLR & WEBHOOK HANDLING

- Separate webhook per vendor & channel
- Map vendor_message_id → smartbeeps_id
- Standardize status across vendors
- Store raw webhook payload
- Fast acknowledgment (return 200 quickly)

Supported events:

SMS: Delivered / Failed

WhatsApp: Sent / Delivered / Read / Failed

RCS: Delivered / Read / Clicked

8. DATABASE STRUCTURE (MINIMUM REQUIRED)

Core Tables:

- users

- clients
- vendors
- routes
- templates
- headers
- channels
- messages
- message_attempts
- message_logs
- dlr_logs
- transactions
- vendor_health_logs

Messages Table Must Include:

- smartbeeps_id
- client_id
- channel (sms/whatsapp/rcs)
- vendor_id
- vendor_message_id
- mobile
- template_id
- route
- cost
- status
- submit_time
- delivery_time

Proper indexing required on:

smartbeeps_id
vendor_message_id
client_id
submit_time

9. BILLING & WALLET SYSTEM

- Prepaid & Postpaid support
 - Wallet deduction on send
 - Channel-based pricing
 - Vendor cost vs client pricing tracking
 - Transaction logs
 - Downloadable billing report
-

10. DEPLOYMENT REQUIREMENTS

-
- Production VPS setup (AWS/DigitalOcean)
 - SSL configuration
 - Environment separation (dev/prod)
 - Redis setup
 - Database optimization
 - Load testing
 - Basic monitoring setup
-

11. TECHNOLOGY STACK (PREFERRED)

Backend: Node.js (NestJS preferred) OR Laravel
Database: PostgreSQL (preferred) or MySQL
Queue: Redis
Frontend: React / Vue / Laravel Blade
Hosting: AWS / DigitalOcean VPS

12. DELIVERABLES

- Fully functional SuperAdmin, Admin & Client panel
 - SMS + WhatsApp + RCS integration
 - Multi-vendor routing engine
 - Queue-based processing system
 - Failover & health monitoring
 - DLR & webhook handling
 - Wallet & billing module
 - Production deployment
 - Source code handover
 - Basic technical documentation
-

IMPORTANT:

This system must be built as a scalable CPaaS foundation capable of handling 1+ crore messages per month in future.

Please share:

- Relevant CPaaS / messaging experience
- Proposed tech stack

- Timeline
- Cost estimate
- Team structure (if agency)

We are looking for a serious long-term technical partner.

Regards,
Smartbeeps